

ME 350: PID Controllers

Joe Saginaw

November 2019

1 Introduction

Before we dive into PID controllers, the field of "controls" should first be discussed. Controls refers to the study and application of making a system behave as desired. While this can be any system, we will focus on the application to *physical systems*. This could be the adjustment of brake fluid pressure to the wheels of a car under heavy braking to prevent them from locking, to the regulation of an output voltage in a DCDC converter. The field of controls is extremely diverse and the concepts studied can be used to both explain, model, and control, physical systems.

Two main areas of control include *feedback* and *feedforward* control. Feedback control is *reactive*, in that some action is calculated from the current or previous *state* of the system. Feedforward control is *proactive*, in that some action is taken based on prior knowledge about how the system will behave. The PID controller is a form of *feedback* controller and is a very powerful tool in the toolbox of a controls engineer. It can be implemented in both analog and digital circuitry with rather few components, is quickly tunable, and thankfully, is somewhat intuitive. This document will discuss the theory and application of PID controllers.

2 Theory

Before we can implement a PID controller, we must first understand how and why they work. This section will be dedicated to the theory, modeling, and pre-implementation analysis of the PID controller.

2.1 But First, A Model

To create a testbed for the PID controller, we will use an example system. We will choose a simple mass-damper (no spring) system, with mass m and damping coefficient B . There will be some time varying force, $F(t)$, acting on the mass. A diagram of the example system is shown below:

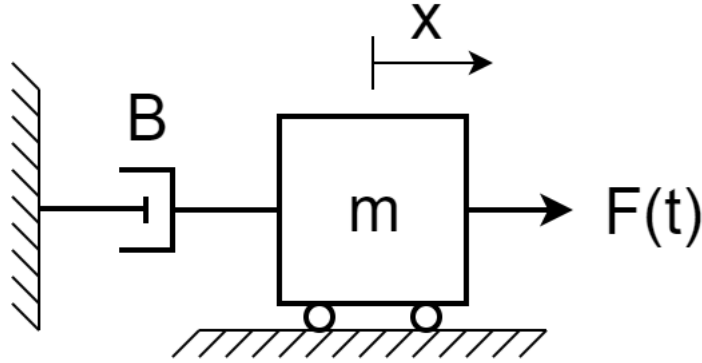


Figure 1: Mass-damper model.

We derive the equations of motion for the mass-damper system:

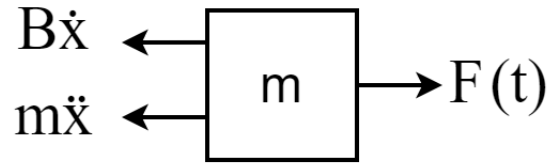


Figure 2: Free-Body Diagram of the mass-damper system.

$$m\ddot{x} + B\dot{x} = F(t) \quad (1)$$

This is a translational model, meaning the mass moves in a linear fashion, but this simple model could be extended to rotary systems as well. An inertia with damping (friction) is very similar to the linkage system designed in ME350. In ourNote that in our simple model we neglect the static, or *coulomb* friction. This is because this type of friction is nonlinear, and is difficult to model accurately. The equations of motion for an inertia J , with rotary damping B , with input torque $\tau(t)$, is shown below:

$$J\ddot{\theta} + B\dot{\theta} = \tau(t) \quad (2)$$

First, lets see what happens when we leave the model with zero initial conditions and zero input:

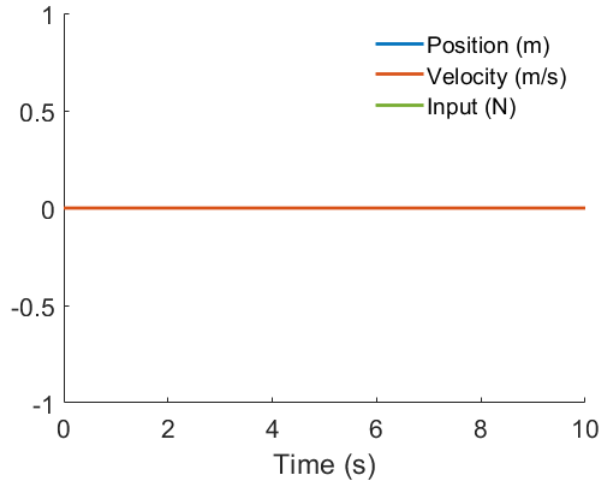


Figure 3: Zero input and zero state response of the mass-damper system.

Nothing happened. Fortunately this was to be expected as there are 1) no external forces on the system and 2) the system has no initial acceleration or velocity. Thus, our system is *stable*. This means in order for the system to stay put, we don't have to do anything. Stable systems tend to be easier to control than *unstable* ones, such as a magnetic levitation device, inverted pendulum, or a fighter jet.

Now let's see what happens again if we impart an initial velocity, $\dot{x}_0$, to the system, again with zero input:

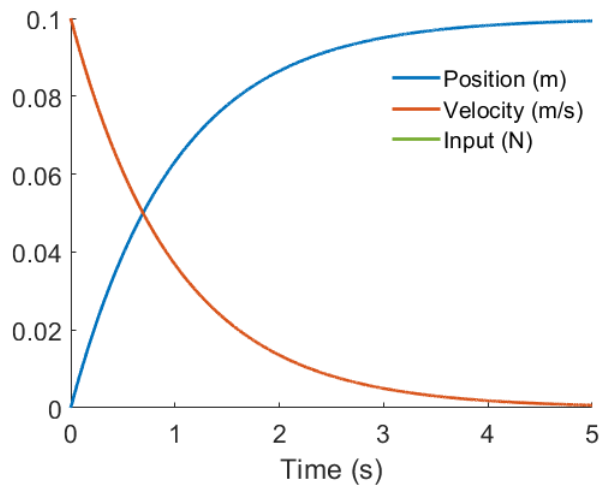


Figure 4: Zero input response of the mass-damper system.

The initial velocity of the system decays away exponentially, the position

increases at a decreasing rate eventually reaching a steady state value, and there is zero input. Note that the values used here for m , B , and $\dot{x}_0$, are not relevant as we are only interested in the qualitative behavior.

Now lets see what happens when there is zero initial velocity and zero initial position, but we impart a small, constant, force to the mass:

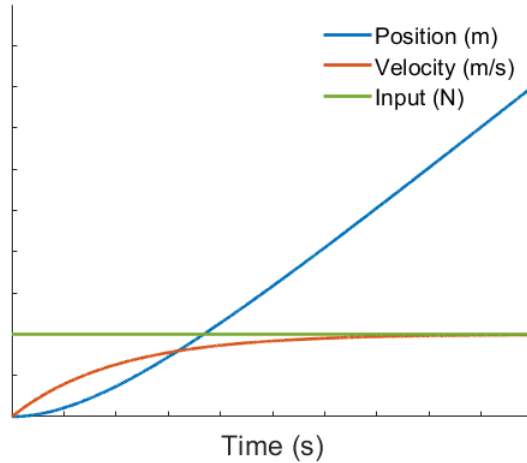


Figure 4: Zero state response of the mass-damper system.

Now that we know what the response of the system is to various initial states and inputs, lets see if we can command the system to achieve a specific position. Specifically, we want the mass to start at $x = 0$ and end at precisely $x = 1$ meter. We will try multiple different forces to try and achieve the desired position:

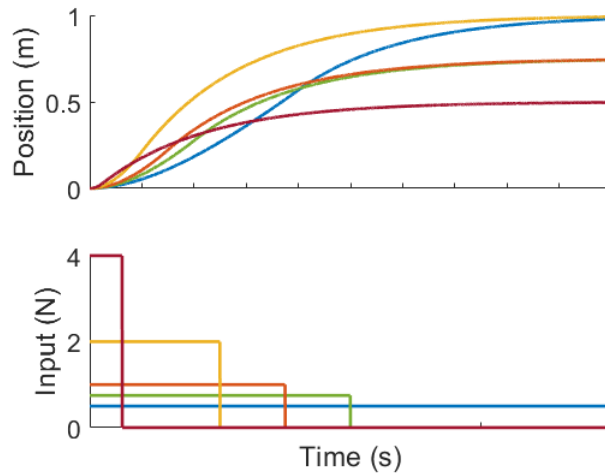


Figure 5: Position response of mass-damper system to various magnitude and duration forces.

As you can see, the 2N and 0.5N pulses all achieved our goal of a steady state position of 1m. This type of control is called *feedforward* control, and usually requires a good model of the system to be accurate. Feedforward control can be extremely powerful under the right set of conditions, but ultimately, has some major drawbacks:

- Requires knowledge of the system before the controller is implemented
- Requires an accurate model of the system to determine proper commands to achieve desired output
- Cannot account for and respond to external forces on the system (this is called *disturbance rejection*)

In the ME350 project, `FRICTION_COMP_VOLTAGE` is an example of feed-forward control, and by testing to see at what voltage the linkage moves, you are measuring parameters in the system. With this knowledge, you can design a command to overcome friction. The drawbacks of feedforward control can be understood if you imagine trying to achieve all 5 target positions with 5 different `FRICTION_COMP_VOLTAGE`s. It's possible, but very difficult and sensitive to changes in the linkage. Lets see what happens to our mass-damper system to the same 2N and 0.5N inputs, but with an increase in damping of 25%:

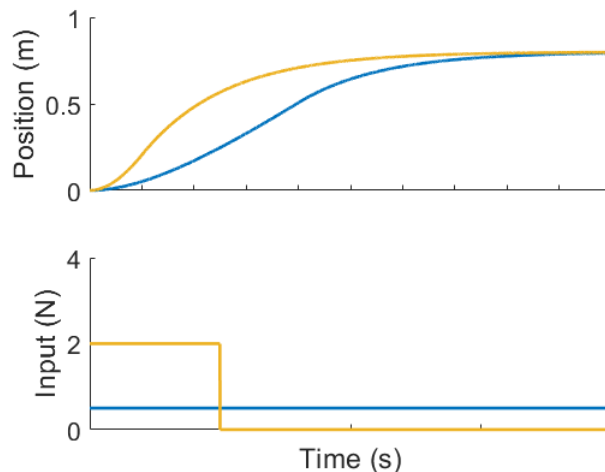


Figure 6: Position response of mass-damper system to various magnitude and duration forces with increased system damping.

The result above shows that if the model changes, or is not determined correctly in the first place, the output will not reach the desired value. Thus we require a different type of control, one that doesn't depend on an accurate model, one that rejects disturbances, and one that doesn't require extensive knowledge of the system before we need to control it. Such a type of control exists, and is called *feedback control*.

2.2 Introduction to Feedback Control

Our main issue with the feedforward only controller from the previous section was it could not account for changes to the system, and also could not reject disturbances. We will now look at a type of control called *feedback control*, which involves measuring the variables we want to control, and taking an action based on the state of these variables.

We won't dive too much into it here, but it is important to understand the concept of *system states*. System states are arbitrarily chosen by the user, but there are some choices that make it easier for the engineer to design a controller for the system. In this case, we will choose $\dot{x}$ and x to be our system states, corresponding to the position and velocity of the mass. With knowledge of these states, we can determine the behavior of the mass over time.

First, we will create a block diagram of the *open-loop system*. The open-loop system is the input to output relationship in the absence of any feedback. Some input will produce some output, that is related only by characteristics of the system:



Figure 7: Block diagram of the open-loop system.

An important term, *Plant*, was introduced here. Plant is common nomenclature for the system we want to control. The plant captures all of the dynamics of the system, and in our case is equal to the mass-damper system we used earlier. We now add a controller:

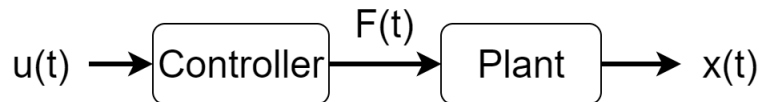


Figure 8: Block diagram of the open-loop system with a controller.

Now we add the critical portion of the block diagram: the *feedback*. In this case we are going to take the output, or the states, and feed it back into the controller. Before we feed it back though, we also subtract it from the input. This creates a signal called the *error signal*, or $e(t)$. This model is called the *closed-loop system*, as we are "closing the loop" between the output and input.

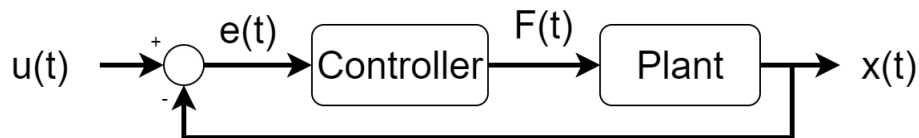


Figure 9: Block diagram of the closed-loop system.

Often, we also include a block dedicated to describing the behavior of the *sensor* that we use to measure the states:

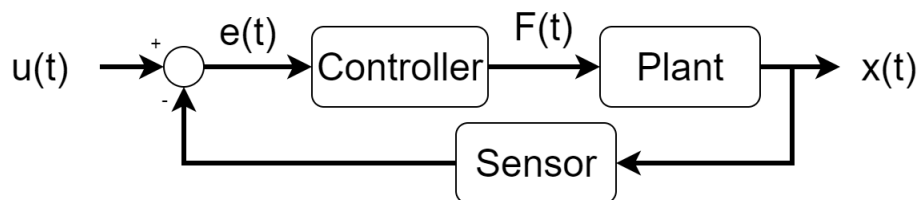


Figure 10: Block diagram of the closed-loop system with a sensor.

Now, in order to determine the relationship between our input and output, we would need to use a *convolution integral* to precisely determine this relationship in the time domain. However, these integrals are messy and the specifics are left to a Signals or Signal Processing course such as EECS 216. The easiest way to analyze this and most control systems is to use Laplace domain analysis, as it allows us to multiply blocks together to obtain a total response. We convert the block diagram into Laplace notation:

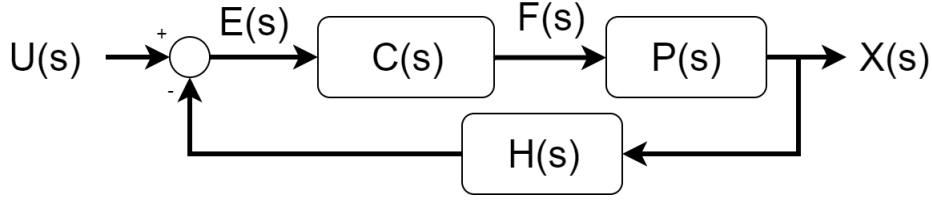


Figure 11: Block diagram of the closed-loop system in the Laplace domain.

We see that instead of time-domain signals such as $u(t)$, all signals and blocks have been converted to the s -domain, the Controller becomes $C(s)$, the Plant becomes $P(s)$, and the Sensor becomes $H(s)$. These various functions of s are called *transfer functions*, as they relate the input of the block to the output of the block. For instance, $C(s)$ relates $E(s)$ and $F(s)$, and since we can multiply Laplace domain signals together, it is specifically:

$$\frac{F(s)}{E(s)} = C(s)$$

Which implies that:

$$E(s) \frac{F(s)}{E(s)} = E(s)C(s) = F(s)$$

This is a very powerful statement as it means that all we need to precisely determine the output of the system is the input and the characteristics of the block that relate the input and the output. We next want to derive a relationship between $U(s)$ and $F(s)$, which are our closed-loop system inputs and outputs, respectively. First, we determine an equation for $X(s)$ in terms of $E(s)$ by multiplying $E(s)$, $C(s)$, and $P(s)$ together:

$$E(s)C(s)P(s) = X(s)$$

In addition, we also know that $E(s)$ is equal to the input $U(s)$, minus the output times the sensor transfer function, $H(s)$:

$$E(s) = U(s) - H(s)X(s)$$

We then substitute our equation for $E(s)$ back into the original equation:

$$[U(s) - H(s)X(s)]C(s)P(s) = X(s)$$

Isolating $X(s)$:

$$U(s)C(s)P(s) = X(s)[1 + H(s)C(s)P(s)]$$

Obtaining the final transfer function relating $X(s)$ to $U(s)$:

$$\frac{X(s)}{U(s)} = \frac{C(s)P(s)}{1 + H(s)C(s)P(s)} \quad (3)$$

Equation (3) represents the *closed-loop transfer function* for a very large number of feedback control architectures. Using Laplace techniques, we successfully related the output of the system to its input that only requires knowledge of the individual block transfer functions.

Lets convert our mass-damper system into a Laplace domain transfer function. Rewriting equation (1):

$$m\ddot{x} + B\dot{x} = F(t) \quad (1)$$

$$L[m\ddot{x} + B\dot{x} = F(t)] = s^2X(s)m + sX(s)B = F(s)$$

$$X(s)(ms^2 + Bs) = F(s)$$

$$\frac{X(s)}{F(s)} = \frac{1}{ms^2 + Bs} \quad (4)$$

Equation (4) represents the transfer function of the plant. For now, we will consider $C(s)$ to be a generic model of a controller so we will leave it as $C(s)$. $H(s)$ will be equal to 1:

$$\frac{X(s)}{U(s)} = \frac{C(s)\frac{1}{ms^2+Bs}}{1 + C(s)\frac{1}{ms^2+Bs}} = \frac{C(s)}{ms^2 + Bs + C(s)}$$

Now that we have a relationship between the input and output of the system and a model for the plant, we can begin to design the controller, $C(s)$

2.3 Proportional Control

In the previous section the error signal, $e(t)$ in the time domain, and $E(s)$ in the s-domain, was defined. This signal is important as it is the input to the controller, and is what the output of the controller will be related to.

The fundamental idea behind a Proportional Controller, or P-Controller, is that the output of the controller is *proportional* to the error signal. We call the constant of proportionality K_p . At the beginning of motion, the input to the system, or the desired position, minus the actual position, is at its greatest. As the system moves toward the desired position, the error signal decreases, and the output of the controller goes decreases proportionally. A block diagram, as well as a Simulink implementation of this controller, are shown below:

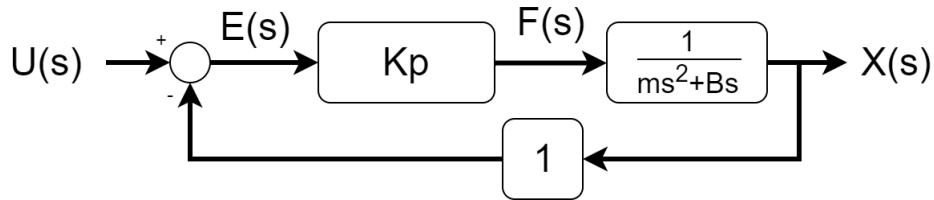


Figure 12: Block diagram of the P-controller.

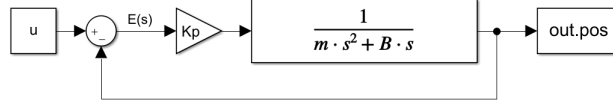


Figure 13: Simulink model of the P-controller.

Deriving a new closed-loop transfer function we arrive at:

$$\frac{X(s)}{U(s)} = \frac{K_p}{ms^2 + Bs + K_p}$$

If we look at what happens under steady state conditions ($t = \infty$, $\lim_{s \rightarrow 0}$), the transfer function becomes $\frac{K_p}{K_p} = 1$, and the output, $X(s)$ exactly equals the input, $U(s)$. Thus, the *steady state error* is equal to zero. Note that if our model had some spring element, which has a linear force to displacement relationship, the steady state error would not be zero.

With a desired position of $x = 1$ meter we simulate the system response:

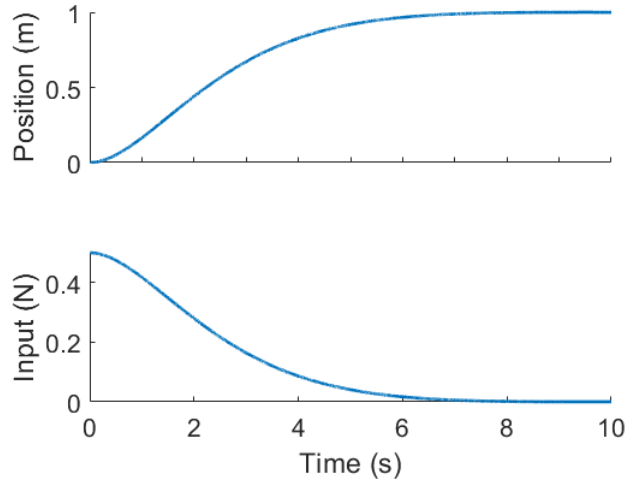


Figure 14: System response with a P-Controller with $K_p = 0.4$.

The position increases at an increasing rate, but then begins to level out to the steady state value. The input is at its maximum when the error is greatest, at $t = 0$. The controller output then decreases as the system gets closer to the desired value and the error decreases. Our open loop system was shown to be sensitive to changes in the model. If we introduce the same condition of a 25% increase in damping:

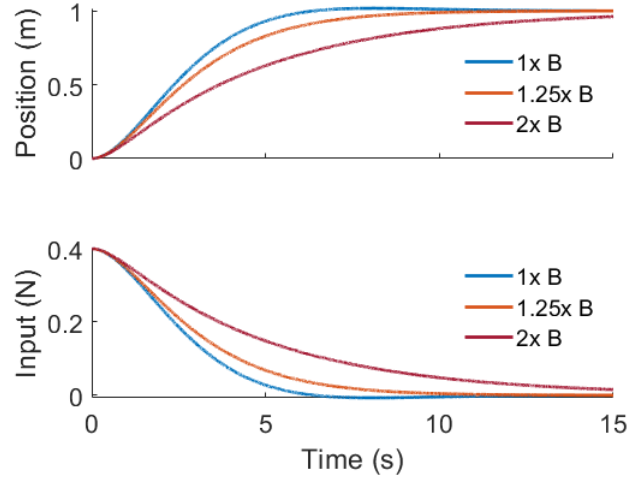


Figure 15: System response with changes in damping with a P-Controller with $K_p = 0.4$.

The controller is still able to achieve the desired output, even with a 2x change in damping! We also observe that the time it takes for the mass to reach the steady state position is quite large. To solve this, let's try increasing K_p to get a larger controller output for the same error:

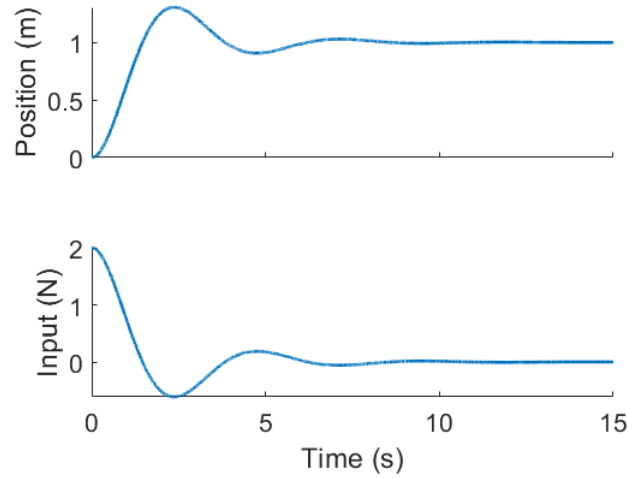


Figure 16: System response with a P-Controller with $K_p = 2$.

With K_p increased to 2, the system is now exhibiting new behavior. The position increases rapidly, as desired, but then goes past the desired value, only to come back the other way past it again. Note that the error signal changes sign

when the mass passes the desired position, which is why the position decreases after a certain value. The bottom plot shows how the force becomes negative for a short while before becoming positive again.

These oscillations are due to the higher K_p value. Since the mass has inertia and the P-controller has no way to "slow-down" the mass, it *overshoots* the desired value. It may seem impossible to have oscillations in a physical system with only a damper and a mass (See Figure 4), but in this case the proportional controller is acting like a spring, causing the oscillations. The error signal is similar to the displacement of the spring, and K_p the "stiffness" of the spring. Intuitively, we would expect the oscillations to grow larger as the spring stiffness increases:

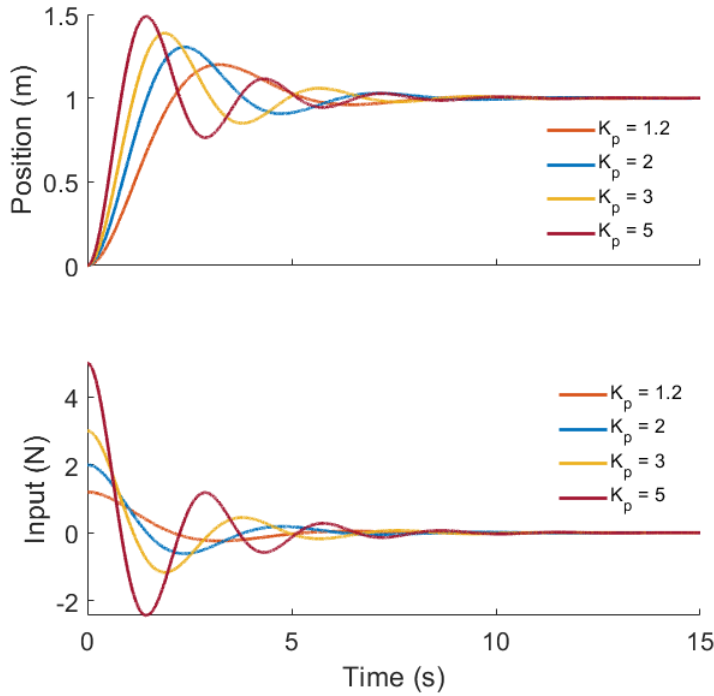


Figure 17: System response with P-Controllers of various K_p s.

As K_p gets larger and the theoretical spring gets "stiffer," the system response becomes faster, but overshoots the desired value by a larger and larger amount. This is a fundamental trade off involved with P-Controllers, balancing *rise time* and *overshoot*.

Our block diagram earlier only had one input, $U(s)$, which we control as the desired position. We will now expand our model to include *external disturbances*, to determine how well our P-Controller *rejects disturbances*. We now include a *disturbance force*, $F_d(t)$, acting on the mass:

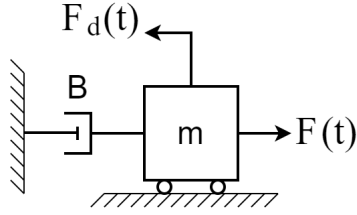


Figure 18: Mass-damper model with a disturbance force.

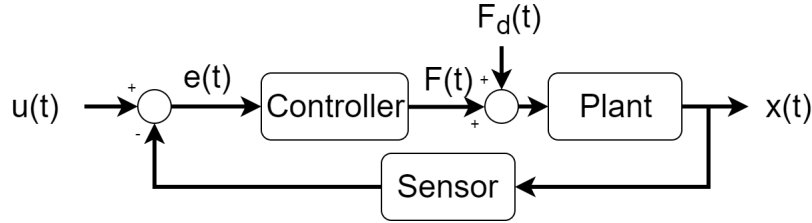


Figure 19: Block diagram of the closed-loop system with a disturbance force.

Using similar math to that we used to derive equation (3), we can determine the transfer function from $F_d(t)$ to $x(t)$, again using Laplace techniques:

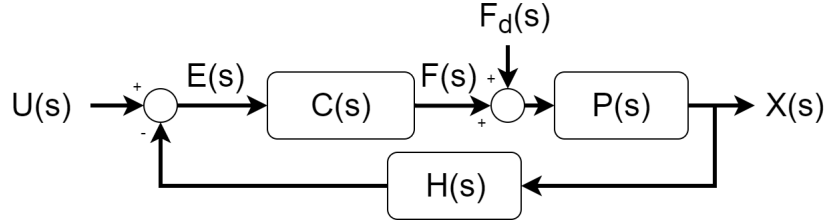


Figure 20: Block diagram of the closed-loop system with a disturbance force.

$$F(s) + F_d(s)]P(s) = X(s)$$

$$F(s) = C(s)E(s)$$

$$E(s) = U(s) - H(s)X(s)$$

$$F(s) = C(s)U(s) - C(s)H(s)X(s)$$

$$C(s)U(s) - C(s)H(s)X(s) + F_d(s)]P(s) = X(s)$$

$$C(s)P(s)U(s) + F_d(s)P(s) = X(s)[1 + H(s)C(s)P(s)]$$

$$X(s) = U(s) \frac{C(s)P(s)}{1 + H(s)C(s)P(s)} + F_d(s) \frac{P(s)}{1 + H(s)C(s)P(s)} \quad (5)$$

Equation (5) represents the complete closed loop transfer function from both the input, $U(s)$, and the disturbance force, $F_d(s)$. Note that the relationship is not a simple $\frac{\text{output}}{\text{input}}$, as there are now multiple inputs and each contribution to the output must be determined individually.

Inputting our plant model for $P(s)$, the P-Controller K_p for $C(s)$, and 1 for $H(s)$:

$$X(s) = U(s) \frac{K_p}{ms^2 + Bs + K_p} + F_d(s) \frac{1}{ms^2 + Bs + K_p}$$

If we observe the condition at steady state, ($t = \infty, \lim_{s \rightarrow 0}$), the first term contributes 1, and the second term contributes $F_d(0) \frac{1}{K_p}$. This would imply that when K_p is large, the second term becomes negligibly small (0), and again the output equals the input, and thus the P-Controller with a large K_p provides good *disturbance rejection*.

Using K_p of 2, and a constant disturbance force of 0.25N, we simulate the system:

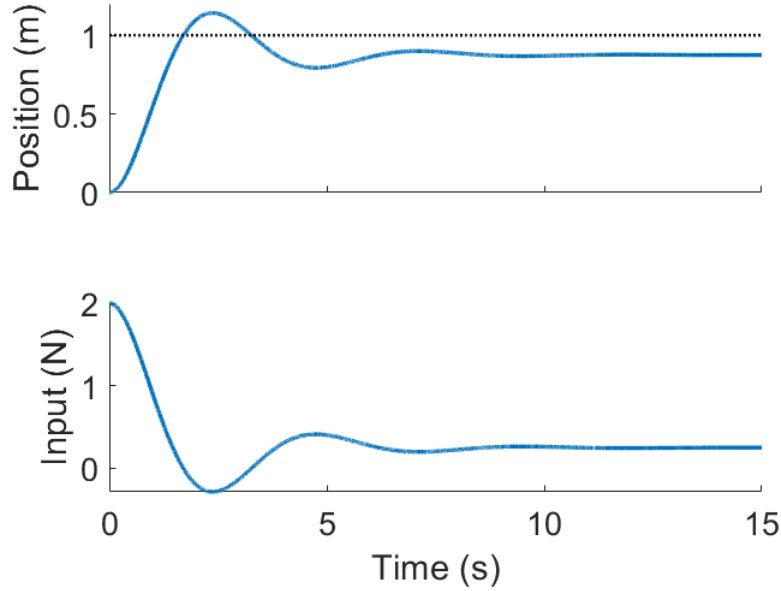


Figure 20: System response with P-Controller with $K_p = 2$ and constant disturbance force of 0.25N.

We see that the system no longer achieves the desired steady state output with the presence of the disturbance force. Earlier, we mentioned that the P-

Controller provides good disturbance rejection when K_p is *large*. Simulating with larger and larger K_p s:

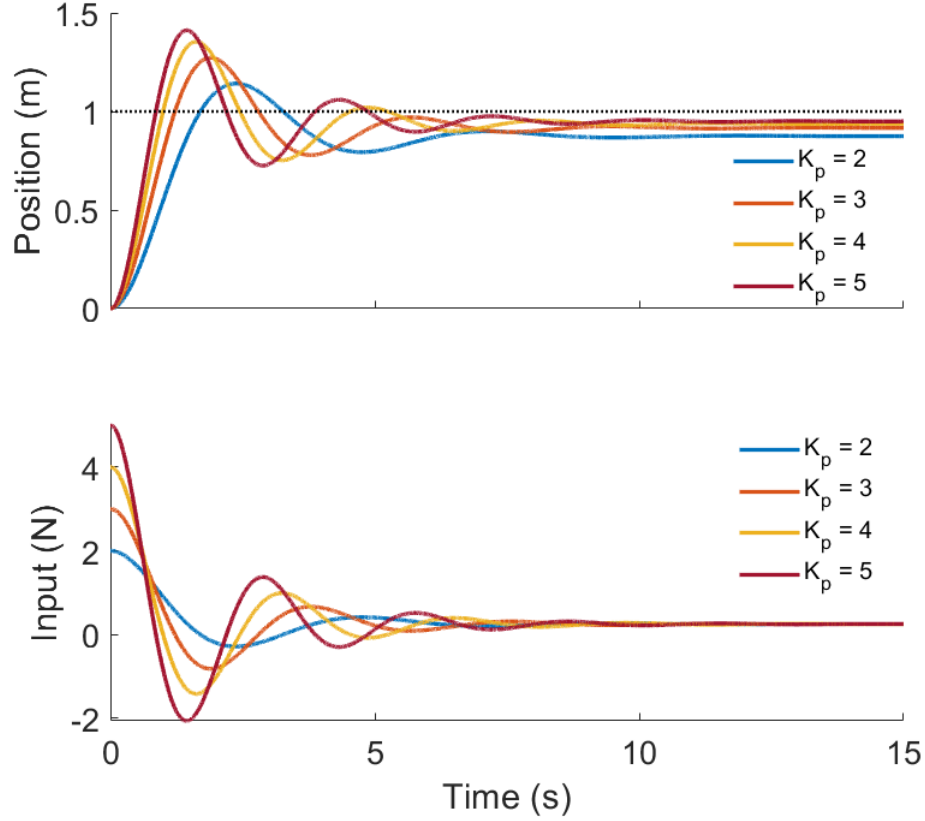


Figure 21: System response with P-Controllers of various K_p s and constant disturbance force of 0.25N.

As K_p increases, the steady state error also decreases. However, we sacrifice overshoot, as the system begins to overshoot more and more. Returning to equation (5) and our substituted values for our system and controller:

$$X(s) = U(s) \frac{K_p}{ms^2 + Bs + K_p} + F_d(s) \frac{1}{ms^2 + Bs + K_p}$$

$$\lim_{s \rightarrow 0} [X(s)] = U(s) + F_d \frac{1}{K_p}$$

We see the effects of F_d are diminished with higher values of K_p . The constant disturbance force is similar to the friction in your linkage, but the difference is that friction only opposes motion, and does not act when the linkage is stationary.

In summary, for P-Controllers:

- P-Controllers can decrease system response time and help reject disturbances
- **Increasing** K_p leads to shorter rise times, lower steady state error, but more overshoot.
- **Decreasing** K_p leads to longer rise times, more steady state error (worse disturbance rejection), and less overshoot.

To help eliminate some of the drawback of the pure P-Controller, we will introduce an additional type of control.

2.4 Derivative Control

The "D" in "PID" stands for Derivative, or Derivative control. Our main issue with the P-Controller was the large overshoot present at high K_p values, which we wanted due to the disturbance rejection and rise time. If the P-Controller acted like a spring, in order to "damp out" the oscillations from high K_p , we would like to add a damper-like element. If the P-Controller produces a command proportional to the error, we desire a controller that produces a command proportional *to the rate of change of the error*. This is called the D-Controller.

It is important to note that if there is no change in the error, there is also no rate of change in the error. Thus, a pure D-Controller will produce no response. It must be paired with a P-Controller for its effects to be appreciated. This is called a PD-Controller.

Our analogous mechanical system now looks like this:

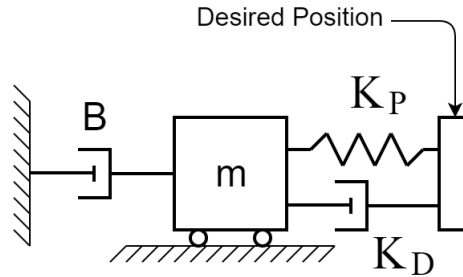


Figure 22: Analogous mechanical system with plant and PD-Controller.

In the time domain, we write the time derivative as $\frac{d}{dt}$, but in the Laplace domain the derivative is equal to a multiplication by s . Thus $\frac{d}{dt}E(t) = sE(s)$. The block diagram now looks like this:

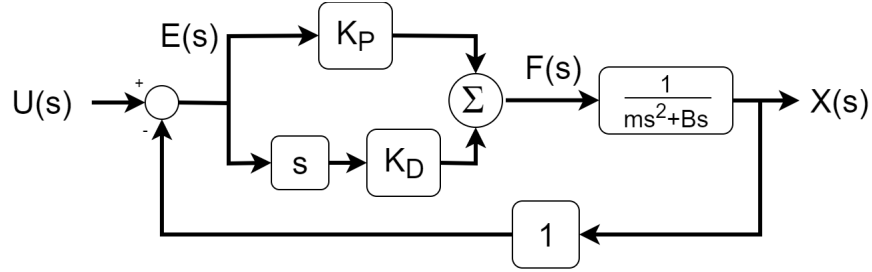


Figure 23: Block Diagram of the PD-Controller in the Laplace domain.

Using similar techniques as before, the closed-loop transfer function becomes:

$$\frac{X(s)}{U(s)} = \frac{K_D s + K_p}{ms^2 + (K_D + B)s + K_p} \quad (6)$$

We can see that the K_D term adds to the damping in the denominator, increasing the effects of damping. A $K_D s$ term also appears in the numerator, which creates effects in the system response called *numerator dynamics*. It is important to note that while it seems K_D is the solution to many of our issues with the P-Controller as we will see in the next plot, it can also make the system more unstable. Different plants react differently in the presence of numerator dynamics, so it is necessary to always check the range over which K_D is *stabilizing* and not *destabilizing*.

We simulate the system with the PD-Controller with $K_p = 5$ and various values of K_D :

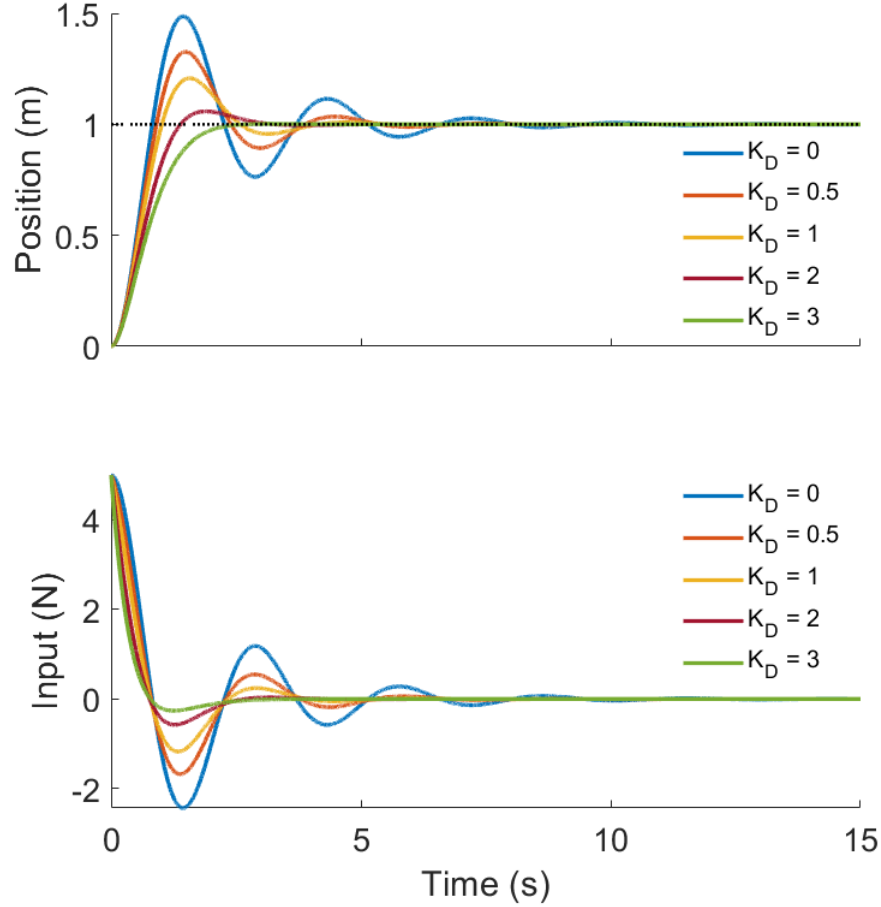


Figure 24: System response with PD-Controllers with $K_p = 5$ and various values of K_D .

Increasing K_D sacrifices a small amount of rise time, but damps out oscillations from the large value of K_p . We also observe that increasing K_D causes the system to obtain the desired value *faster* than with lower values. The time it takes the system to reach the desired value is called the *settling time*. A metric for settling time is the time it takes for the system to "settle" into a bound of $\pm 2\%$ of the desired value.

If we add back a disturbance force of 0.5N and keep a $K_p = 5$ and $K_D = 2$ and re-simulate:

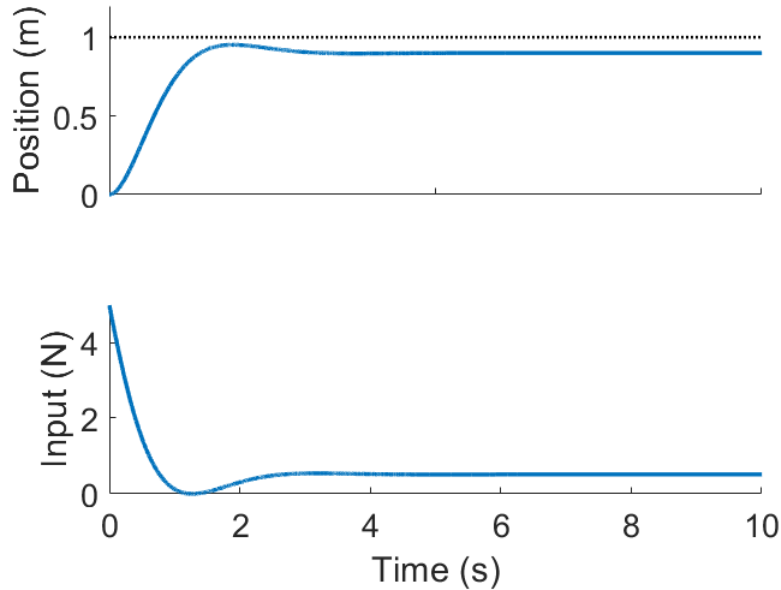


Figure 25: System response with a PD-Controller with $K_p = 5$, $K_D = 2$, and a constant disturbance force of 0.5N.

We see that the addition of the D-Controller was unable to eliminate the steady-state error. In general, the D-Controller will not effect the steady-state error, as the controller is inherently based on changes in the system and will thus have no effect on steady-state behavior.

In summary, for D-Controllers:

- Will not produce any output without a complementary P-Controller
- The addition of a derivative term can damp out oscillations and reduce overshoot, reduce settling time, and slightly increase rise times. It does not effect steady-state error.
- **Increasing** K_D leads to less overshoot, shorter settling times, and slightly longer rise times.
- **Decreasing** K_D leads to more overshoot, longer settling times, and slightly shorter rise times.

We have addressed most of the transient behavior of the system with the PD-Controller, but the last issue with our system is the steady-state error. To address the steady-state error, we introduce an additional type of controller.

2.5 Integral Control

The "I" in "PID" stands for Integral, or Integral control. Our main issue with the PD-Controller was the steady-state error in the presence of disturbances

due our inability to make K_p very large. Thus, we seek a controller to eliminate an offset between our current position and desired position, no matter the input required to obtain the final position. This controller is called the I-Controller. It creates a command that is proportional to the *integral of the error over time*. We will pair it with the P- and D-Controllers to form a *PID Controller*. While we implement the I-Controller to eliminate steady-state error, it also effects the transient response. We will observe how it effects the overall system response below.

Unfortunately, the I-Controller has no intuitive mechanical analog, so we will not update our system to show the I-Controller.

In the time domain, we write the integral with respect to time as $\int_0^t e(t)dt$, but in the Laplace domain the integral is equal to a division by s . Thus $\int_0^t e(t)dt = \frac{E(s)}{s}$. The block diagram now looks like this:

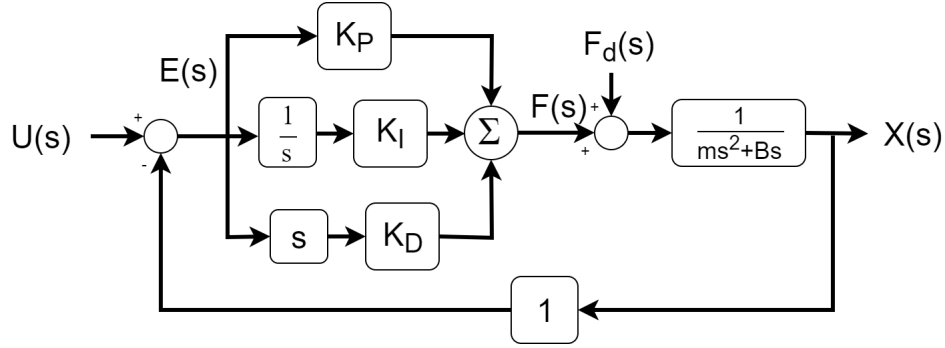


Figure 26: Block Diagram of the PID-Controller in the Laplace domain.

Using similar techniques as before, the closed-loop transfer function becomes:

$$X(s) = U(s) \frac{K_d s + K_p + \frac{K_I}{s}}{ms^2 + (B + K_d)s + K_p + \frac{K_I}{s}} + F_d(s) \frac{1}{ms^2 + (B + K_d)s + K_p + \frac{K_I}{s}}$$

Multiplying both the top and bottom of both transfer function by s :

$$X(s) = U(s) \frac{K_d s^2 + K_p s + K_I}{ms^3 + (B + K_d)s^2 + K_p s + K_I} + F_d(s) \frac{s}{ms^3 + (B + K_d)s^2 + K_p s + K_I} \quad (7)$$

To see what happens at steady-state, we take the limit as $t \rightarrow \infty$, or $\lim_{s \rightarrow 0}$:

$$\lim_{s \rightarrow 0} [X(s)] = U(s) + 0$$

This tells us that no matter what value of K_I we choose, the desired position will always be achieved, and the steady-state error will be zero.

Let's simulate the system with the PID-Controller with $K_p = 5$, $K_D = 2$, various values of K_I , and a disturbance force of 1N:

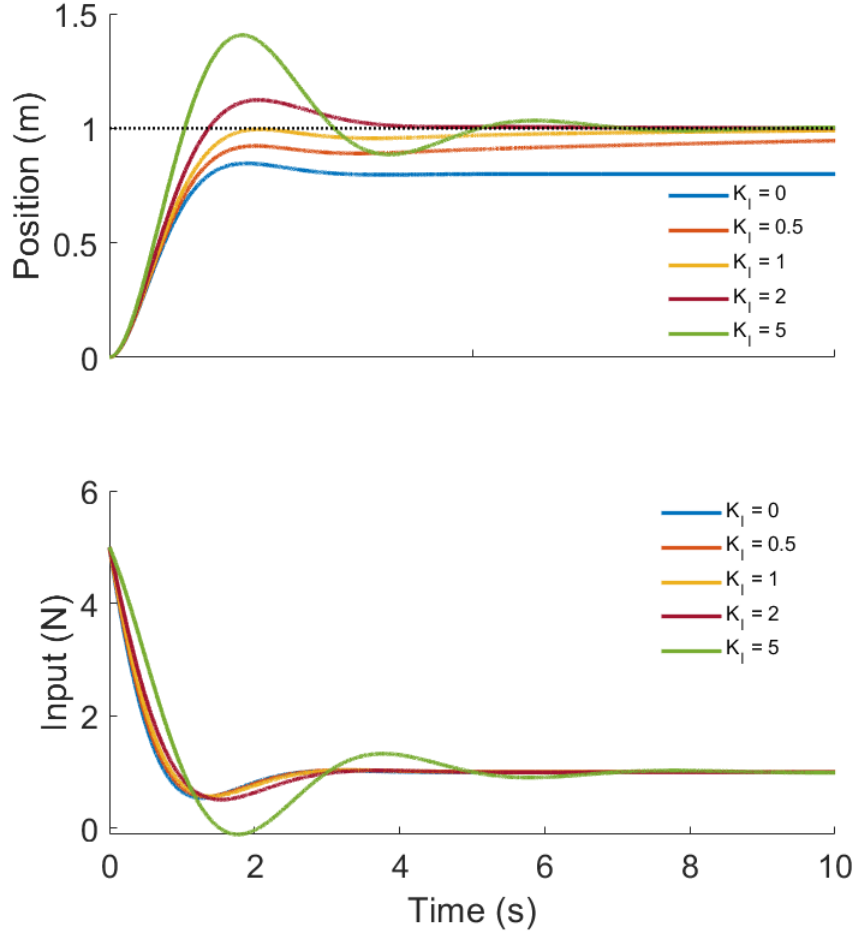


Figure 27: System response with PID-Controllers with $K_p = 5$, $K_D = 2$, various values of K_I , and a constant disturbance force of 1N.

As expected, all nonzero values of K_I yield zero steady-state error, with various times taken to achieve the desired position. Another interesting observation is that increasing K_I also serves to increase overshoot and oscillations, and also decrease rise time. To better understand why this happens, let's look at the different contributions to the total input by the P-, I-, and D-Controllers:

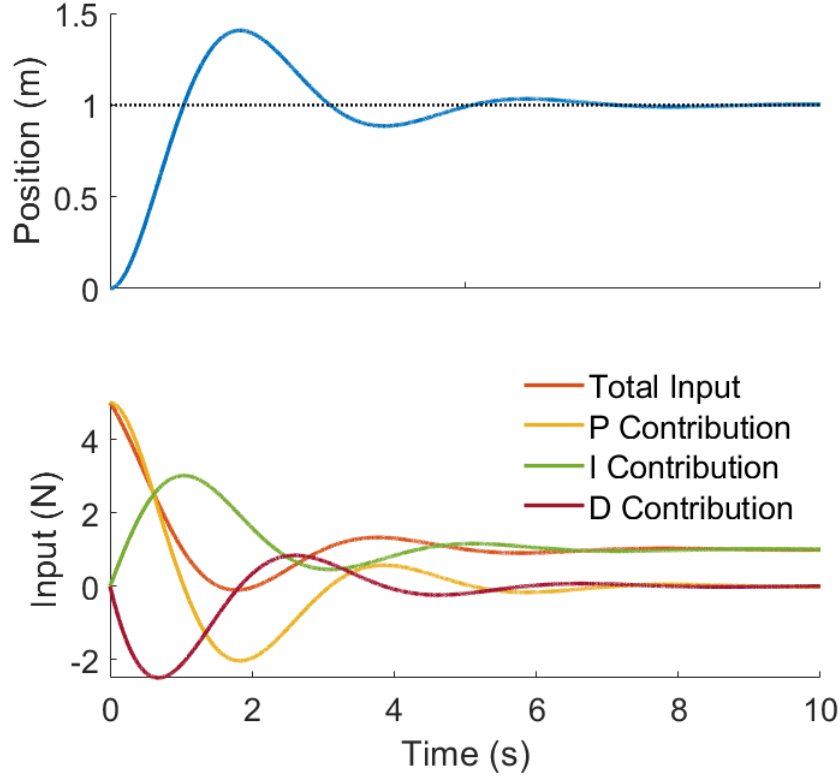


Figure 28: System response and contributions to total input by the different controllers with PID-Controller with $K_p = 5$, $K_D = 2$, $K_I = 5$, and a constant disturbance force of 1N.

In the bottom plot, we see the contributions to the total controller command by the P, I, and D terms. In the initial transient phase, the P and I controllers have a positive input, making the rise time faster and overshoot higher. The D term has a negative contribution, as the *rate of change of the error* is negative, and serves to slow down the response. At around $t = 6$ s, the P and D contributions are seen to decay to zero, with the I contribution being the only part of the controller helping to reject the disturbance. This is because the error integrates to a value that directly counteracts the disturbance force, and then stops integrating once the error becomes zero at the desired position.

In summary, for I-Controllers:

- Usually paired with a complementary P-Controller
- The addition of a integral term can eliminate steady state error, increase overshoot, and decrease rise time, and increase settling time (due to oscillations).

- **Increasing** K_I leads to more overshoot, lower rise times, and faster rejection of any disturbance or steady-state error.
- **Decreasing** K_I leads to less overshoot, longer rise times, and slower rejection of any disturbance or steady-state error.

2.6 PID-Controllers in Summary

PID Controllers are very powerful feedback controllers and their effects on our simple mass-damper system were demonstrated above. A summary of the various effects of K_p , K_D , and K_I is shown below:

Table 1: Effects of *increasing* each of the PID parameters.

Parameter	Rise Time	Overshoot	Settling Time	Steady-State Error
K_p	Decrease	Increase	Increase	Decrease
K_D	Increase	Decrease	Decrease	No Effect
K_I	Decrease	Increase	Minimal Effect	Eliminate

To further illustrate how much we improved the system response, we simulate the system again with $K_p = 20$, $K_D = 3$, $K_I = 8$, and no disturbance force:

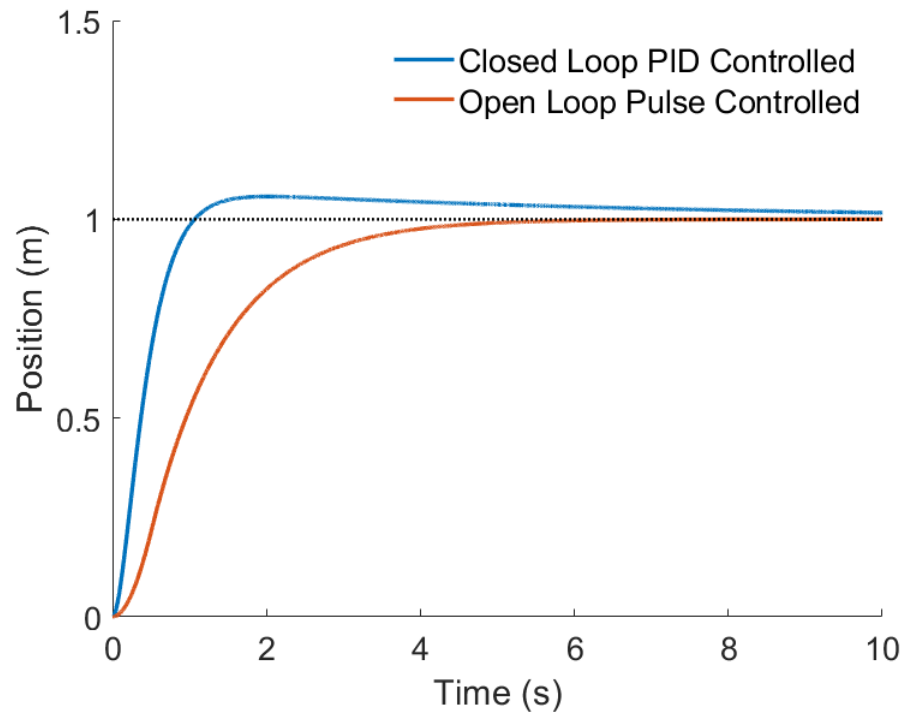


Figure 29: PID Controlled Closed Loop Response vs Open Loop feedforward response.

The PID Controller decreases rise time and settling time compared to the open-loop response, and has zero steady state error. If we added a disturbance force or our model changed, the open-loop controller would fail to reach the desired value. The benefits of the PID Controller become even more apparent when a complex input signal is commanded, such as this sine-wave square-wave combination:

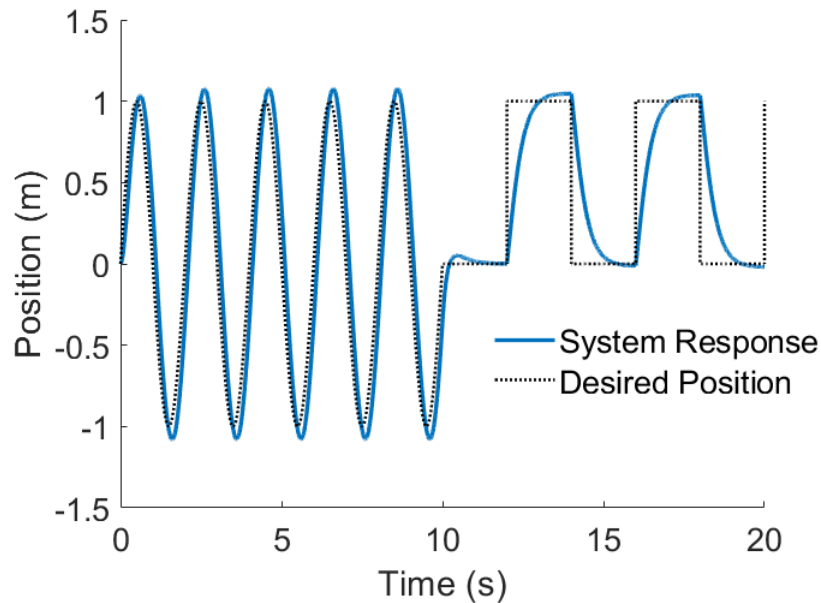


Figure 30: System response with PID Controller to complex input signal.

The controller exhibits good *command tracking*. It would be hard to exactly know the commands to achieve the desired position to successfully implement feedforward control.

Many tools and methods exist to tune PID controllers, including but not limited to:

- Ziegler-Nichols tuning method
- Modified Ziegler-Nichols tuning method
- Root Locus analysis methods
- Nyquist frequency domain analysis
- Bode frequency domain analysis

Many of these methods are easily implementable in software such as MATLAB or Simulink, and allow for easy model and controller building and tuning.

3 References and Further Reading

Many resources exist in print and online relating to model generation, controller design and tuning, and controller analysis. A few that were referenced in the creation of this document are listed below:

- Kuo and Golnaraghi, *Automatic Control Systems, 9th edition*, 2010
- Palm, *System Dynamics, 3rd edition*, 2013
- Control resources created at Michigan: <http://ctms.engin.umich.edu/CTMS>

There are also numerous undergraduate and graduate controls classes offered at the University of Michigan. A small subset is listed below:

- ME360: Modeling, Analysis, and Control of Dynamic Systems
- ME461: Automatic Control (cross-listed with EECS460: Automatic Control)
- ME560: Modeling Dynamic Systems
- ME564: Linear Systems Theory
- EECS461: Embedded Control Systems